Migración a TypeScript de FamiFinanzas: aplicación web para la gestión del presupuesto familiar

Samuel Salamanca
Escuela de Ingeniería de Sistemas y Computación
Universidad Pedagógica y Tecnológica de Colombia
Tunja, Colombia

Rafael Cristancho
Escuela de Ingeniería de Sistemas y Computación
Universidad Pedagógica y Tecnológica de Colombia
Tunja, Colombia

***Resumen*—Este documento describe la migración a TypeScript de FamiFinanzas, la aplicación web de gestión de gastos, ingresos y presupuesto familiar desarrollada en el Taller 2 del primer corte con Node.js, Express, EJS, JWT y SQLite. La migración abarcó tres frentes: (i) la reestructuración del modelo en una jerarquía de clases con interfaces, clases abstractas, genéricos, enumeraciones y encapsulamiento; (ii) el reemplazo de todos los scripts JavaScript del backend por archivos `.ts` transpilados con `tsc` a JavaScript limpio en la carpeta `dist/`; y (iii) un nuevo frontend escrito en TypeScript que se transpila a módulos ES nativos (`public/js/*.js`) y agrega validación en vivo, simulación del presupuesto, un gráfico interactivo y tablas ordenables. Los contratos de datos (DTO) se comparten entre servidor y cliente mediante un archivo de declaraciones, de modo que un cambio en la API se detecta en tiempo de compilación en ambos lados. La API REST conserva su comportamiento: las 29 pruebas de integración originales pasan sin modificar su lógica, junto con 12 pruebas nuevas, y el rendimiento se mantiene por debajo de 10 ms (p95) con 10 000 gastos.**

***Palabras clave*—TypeScript, migración de software, programación orientada a objetos, Node.js, Express, transpilación, UML.**

I. Introducción

JavaScript es el lenguaje de la web, pero su tipado dinámico traslada a tiempo de ejecución errores que podrían detectarse antes: propiedades mal escritas, parámetros con la forma equivocada o valores `undefined` que se propagan. Gao, Bird y Barr estimaron que un sistema de tipos estático como el de TypeScript habría detectado cerca del 15 % de los errores públicos analizados en proyectos JavaScript [3]. TypeScript es un superconjunto de JavaScript que agrega anotaciones de tipo opcionales, interfaces, genéricos y modificadores de acceso, y que se transpila a JavaScript estándar [1], [2].

En el Taller 2 se construyó *FamiFinanzas*, una aplicación web para que una familia registre sus gastos e ingresos, defina presupuestos mensuales por categoría y reciba alertas al alcanzar el 80 % y el 100 % de cada presupuesto. El objetivo de este taller es migrar esa aplicación a TypeScript, aprovechando la migración para resaltar las ventajas del lenguaje: un modelo orientado a objetos explícito, backend tipado de extremo a extremo y vistas del cliente implementadas en TypeScript.

El documento se organiza así: la Sección II resume el sistema de partida; la Sección III describe el proceso de migración con fragmentos comparativos de código JavaScript y TypeScript; la Sección IV presenta los modelos UML; la Sección V reporta la verificación y la Sección VI concluye.

Taller TypeScript — Electiva II (60) 8108277, periodo académico II-2026. Universidad Pedagógica y Tecnológica de Colombia.

Tabla I
Requisitos funcionales de FamiFinanzas

ID	Requisito
RF-01	Registrar familia, administrador y 8 categorías iniciales
RF-02	Iniciar sesión con correo y contraseña; emitir JWT (2 h)
RF-03	Gestionar miembros de la familia (administrador)
RF-04	Gestionar categorías; desactivar si tienen gastos
RF-05/06/07	Registrar, filtrar/paginar y editar/eliminar gastos
RF-08	Gestionar ingresos
RF-09	Definir presupuestos mensuales por categoría
RF-10	Alertar al 80 % y al superar el presupuesto
RF-11	Resumen mensual por categoría y por miembro
RF-12	Exportar gastos filtrados a CSV
RF-13	Consultar perfil y cambiar contraseña
RF-14	Autorización por rol y aislamiento entre familias

II. Sistema de partida (Taller 2)

FamiFinanzas expone una API REST versionada (`/api/v1`) documentada con OpenAPI/Swagger y protegida con JSON Web Tokens (JWT) [5], además de un front-end renderizado en el servidor con plantillas EJS que guarda el mismo JWT en una cookie *httpOnly*. La persistencia usa el módulo `node:sqlite` de Node.js 22 [6] y la validación de entradas la biblioteca Zod [7]. La arquitectura sigue capas: rutas (controladores), servicios (reglas de negocio), repositorios (SQL parametrizado) y base de datos. La Tabla I resume los requisitos funcionales, que la migración conserva íntegramente.

El código JavaScript original tenía limitaciones que motivaron la migración: los repositorios y servicios eran objetos literales con lógica duplicada (por ejemplo, `gastoService` e `ingresoService` repetían `obtener` y `eliminar`); la clase abstracta `Movimiento` existía en el diagrama de clases pero no en el código; los estados (`'ADMIN'`, `'EXCEDIDO'`...) eran cadenas sueltas; y el hash de la contraseña se ocultaba "de memoria" con una función `sinHash`. El frontend era un único archivo `app.js` de 25 líneas.

III. Proceso de migración

III-A. Estrategia

La migración se ejecutó de forma incremental y guiada por las pruebas existentes, en cinco fases:

1. **Configuración**: instalación de `typescript` y de las definiciones `@types/*`, y creación de dos proyectos de compilación (backend y cliente) que comparten un `tsconfig.base.json`.
2. **Contratos**: definición de los DTO de la API en `src/shared/contratos.d.ts`.
3. **Dominio**: creación de la jerarquía de clases (Sección III-C).
4. **Backend**: conversión de configuración, base de datos, repositorios, servicios, middlewares y rutas a `.ts`.
5. **Frontend**: implementación de las vistas del cliente en TypeScript (`client/`).

El código JavaScript original se conservó en `legacy-js/` como referencia para la comparación.

III-B. Configuración del compilador

El Listado 1 muestra la configuración común. Se activó el modo `strict` (incluye `strictNullChecks` y `noImplicitAny`), `noImplicitOverride` para exigir la palabra `override` al sobrescribir métodos y `verbatimModuleSyntax`, que obliga a importar los tipos con `import type` para que el JavaScript generado no contenga importaciones innecesarias. El backend usa `module: NodeNext` y emite en `dist/`; el cliente usa `module: ES2022`, las bibliotecas `DOM` y emite en `public/js/` [4].

```
{ "compilerOptions": {
    "target": "ES2022", "strict": true,
    "noImplicitOverride": true, "noImplicitReturns": true,
    "noUnusedLocals": true, "noUnusedParameters": true,
    "isolatedModules": true, "verbatimModuleSyntax": true,
    "sourceMap": true } }
// tsconfig.json (backend):  module NodeNext, lib ES2022,
//   rootDir ".", outDir "dist"   -> dist/src, dist/tests
// tsconfig.client.json:     module ES2022, lib DOM,
//   rootDir "client", outDir "public/js"
```

Listado 1. tsconfig.base.json (opciones comunes) y diferencias por proyecto

Los *scripts* de `npm` quedaron así: `build` ejecuta ambos proyectos; `start` compila (`prestart`) y arranca `node dist/src/server.js`; `test` compila y ejecuta las pruebas transpiladas con el ejecutor nativo `node --test`; `typecheck` verifica tipos sin emitir archivos.

III-C. Jerarquía de clases y elementos de POO

Los modelos se reescribieron como clases (Fig. 10). La raíz es la clase abstracta genérica `Entidad<TDTO>`, que fija el identificador inmutable y obliga a cada subclase a implementar `toJSON()` con la forma exacta de su DTO. `Movimiento<TDTO>`, que en JavaScript solo existía en el diagrama, es ahora una clase abstracta de la que heredan `Gasto` e `Ingreso`; contiene la regla de propiedad RF-07 (`puedeSerModificadoPor`) y el método abstracto `tipo`, resuelto por polimorfismo. Se definieron las interfaces `Serializable`, `Modificable`, `ExportableCsv` y `Actor`; las enumeraciones `Rol`, `MetodoPago` y `EstadoPresupuesto`; y el objeto de valor inmutable `EjecucionPresupuesto`, con constructor privado y método de fábrica estático, que reemplaza a la función suelta `estadoPresupuesto()`.

El encapsulamiento se reforzó con los campos privados de ECMAScript [10]: el hash de la contraseña de `Usuario` es `#passwordHash`, por lo que no aparece en `JSON.stringify`, en el operador de propagación ni en `Object.keys` (Listado 2). Los errores se modelaron como jerarquía (Fig. 12a): cada subclase de `AppError` fija su código HTTP, lo que elimina códigos escritos a mano (Fig. 1).

```
export class Usuario extends Entidad<UsuarioDTO>
                     implements Actor {
  readonly rol: Rol;                  // enum, no string
  readonly #passwordHash: string | undefined;
  get hashContrasena() { return this.#passwordHash; }
  esAdmin(): boolean { return this.rol === Rol.ADMIN; }
  override toJSON(): UsuarioDTO { /* sin el hash */ }
}
// Record<Enum, string> exige una etiqueta por cada valor:
export const NOMBRES_METODO: Readonly<Record<MetodoPago,
  string>> = { [MetodoPago.EFECTIVO]: 'Efectivo', ... };
```

Listado 2. Encapsulamiento con campo privado y tipos de enumeración

III-D. Backend: reemplazo de los scripts JavaScript

Los 25 archivos `.js` de `src/` se reemplazaron por archivos `.ts`. Los cambios más representativos fueron:

Repositorios genéricos. Se creó la clase abstracta `RepositorioBase<TEntidad, TFila>` (patrón *Repository* [8]) que centraliza el acceso a `node:sqlite` y exige implementar `mapear(fila)`. Cada tabla declara la interfaz de su fila (`FilaGasto`, en *snake_case*) y el repositorio devuelve objetos de dominio. Gastos e ingresos comparten `MovimientoRepository<T, TFila>`, que implementa una sola vez los filtros por fecha y miembro, la paginación y el aislamiento por familia; `GastoRepository` solo agrega el filtro por categoría sobrescribiendo un método *gancho* (Fig. 2).

Servicios con herencia e inyección de dependencias. `MovimientoService<T, R>` es abstracta y genérica; `GastoService` e `IngresoService` heredan `listar`, `obtener`, `eliminar` y `verificarPropiedad`. Todas las dependencias se reciben por el constructor y se conectan en una raíz de composición (`contenedor.ts`), lo que eliminó las importaciones circulares entre servicios de la versión JavaScript.

Validación y tipos inferidos. Los esquemas Zod se mantuvieron, pero ahora reutilizan los `enum` del dominio (`z.nativeEnum`) y de ellos se derivan los tipos con `z.infer`; la función `validar` es genérica y su tipo de retorno se deduce del esquema (Listado 3). Así, la validación en ejecución y el tipo estático nunca se desincronizan.

```
export type EntradaGasto = z.infer<typeof esquemas.gasto>;
// { monto: number; fecha: string; categoriaId: number;
//   descripcion: string; metodoPago: MetodoPago }
export function validar<E extends z.ZodTypeAny>(
    esquema: E, datos: unknown): z.output<E> { ... }
const datos = validar(esquemas.gasto, req.body);
gastoService.crear(actor(req), datos); // EntradaGasto
```

Listado 3. Tipos derivados de la validación y validador genérico

```
export class AppError extends Error {
  constructor(status, codigo, mensaje, detalles) {
    super(mensaje);
    this.status = status;
    this.codigo = codigo;
    this.detalles = detalles;
  }
}
export const noEncontrado = (recurso) =>
  new AppError(404, 'NO_ENCONTRADO',
               `${recurso} no encontrado`);
export const conflicto = (mensaje) =>
  new AppError(409, 'CONFLICTO', mensaje);
// En otro servicio, el código se repetía a mano:
throw new AppError(409, 'CONFLICTO', 'La familia...');
```

Antes (JS) — src/utils/errores.js

```
export class AppError extends Error {
  constructor(
    readonly status: number,
    readonly codigo: string,
    mensaje: string,
    readonly detalles?: DetalleErrorDTO[],
  ) {
    super(mensaje);
    this.name = new.target.name;
  }
  aRespuesta(): ErrorApiDTO { ... }
}
export class NoEncontradoError extends AppError {
  constructor(recurso: string) {
    super(404, 'NO_ENCONTRADO', `${recurso} no encontrado`);
  }
}
export class ConflictoError extends AppError { ... }
```

Después (TS) — src/utils/errores.ts

Figura 1. Snapshot comparativo de la migración de los errores: de funciones fábrica a una jerarquía de clases con propiedades de parámetro `readonly`.

```
const mapear = (g) => g && {
  id: g.id, monto: g.monto, fecha: g.fecha,
  metodoPago: g.metodo_pago,
  categoria: { id: g.categoria_id, ... },
  usuario: { id: g.usuario_id, ... },
};
function filtros(familiaId, { desde, hasta,
                 categoriaId, usuarioId }) { ... }
export const gastoRepository = {
  buscar(id, familiaId) {
    return mapear(db.prepare(`${SELECT} WHERE ...`)
                    .get(id, familiaId));
  },
  listar(familiaId, f) { ... },    // duplicado en
  eliminar(id, familiaId) { ... },// ingresoRepository
};
```

Antes (JS) — gastoRepository.js

```
interface FilaGasto {
  id: number; monto: number; fecha: string;
  metodo_pago: MetodoPago; categoria_id: number; ...
}
export class GastoRepository
    extends MovimientoRepository<Gasto, FilaGasto> {
  protected override readonly tabla = 'gastos';
  protected override readonly alias = 'g';
  protected override mapear(g: FilaGasto): Gasto {
    return new Gasto({ id: g.id, monto: g.monto, ... });
  }
  protected override filtrosAdicionales(
      { categoriaId }: CriteriosMovimiento): ClausulaWhere {
    return categoriaId ? { where: 'g.categoria_id = ?',
      params: [categoriaId] } : { where: '', params: [] };
  }
  // buscar, listar, listarTodos y eliminar se heredan
}
```

Después (TS) — GastoRepository.ts

Figura 2. Snapshot comparativo de un repositorio: de objeto literal sin tipos a clase que hereda de un repositorio genérico. La propiedad `tabla` solo admite `'gastos' | 'ingresos'` (tipo unión de literales).

Express tipado. Se amplió el tipo `Request` de Express mediante *module augmentation* para declarar `req.usuario`, y la función `actor(req)` devuelve el usuario autenticado o lanza un 401, en lugar de propagar un `undefined` silencioso. Para el front-end EJS se definió la interfaz `Vistas`, que asocia cada plantilla con los datos que necesita; la función genérica `renderizar<K extends keyof Vistas>` hace que olvidar una variable de la vista sea un error de compilación (Listado 4). Las respuestas usan el operador `satisfies` para comprobar que cumplen el contrato sin perder el tipo inferido.

```
declare global { namespace Express {
  interface Request { usuario?: Usuario } } }
export interface Vistas {
  'pages/login': Base & { aviso: string | null;
                          error: string | null; email:
    string };
  'pages/gastos': Base & { resultado: Pagina<Gasto>;
     filtro: FiltroMovimientos; qs: string; ... };
}
export function renderizar<K extends keyof Vistas>(
  res: Response, vista: K, datos: Vistas[K], status =
    200) {...}
```

Listado 4. Ampliación de Express y render tipado de vistas EJS

Durante la conversión el compilador reportó, por ejemplo, que el arreglo construido para `URLSearchParams` era `string[][]` y no la tupla `[string, string][]` que exige la API del DOM; en JavaScript ese tipo de inconsistencias pasa desapercibido hasta que falla en producción:

```
src/routes/web.ts(94,23): error TS2345: Argument of type 'string[][]'
  is not assignable to parameter of type ... Iterable<[string, string]>
```

III-E. Proceso de transpilación

El comando `npm run build` invoca a `tsc` dos veces. Para el backend, cada archivo `src/**/*.ts` produce un `dist/src/**/*.js` con su *source map*; para el cliente, `client/**/*.ts` produce `public/js/**/*.js`. Al apuntar a ES2022, el JavaScript generado conserva las clases, los campos de clase y los campos privados nativos; solo se eliminan las anotaciones de tipo, las interfaces, los modificadores `private`/`readonly`/`override` y las importaciones de solo tipos. El resultado es código limpio y legible (Fig. 3). Los archivos `.d.ts` (contratos y ampliación de Express) no generan JavaScript.

```ts
import { Movimiento, type DatosMovimiento,
         type TipoMovimiento } from './Movimiento.js';
import type { MetodoPago } from './enums.js';
import type { ExportableCsv } from './interfaces.js';
import type { GastoDTO, ReferenciaCategoriaDTO }
  from '../shared/contratos.js';

export class Gasto extends Movimiento<GastoDTO>
                   implements ExportableCsv {
  static readonly ENCABEZADO_CSV = ['ID', 'Fecha', ...] as
     const;
  readonly metodoPago: MetodoPago;
  readonly categoria: ReferenciaCategoriaDTO;
  constructor(datos: DatosGasto) {
    super(datos);
    this.metodoPago = datos.metodoPago;
    this.categoria = datos.categoria;
  }
  override get tipo(): TipoMovimiento { return 'GASTO'; }
  override toJSON(): GastoDTO {
    return { ...this.camposBase(),
             metodoPago: this.metodoPago,
             categoria: this.categoria };
  }
}
```

Fuente — src/dominio/Gasto.ts

```ts
import { Movimiento } from './Movimiento.js';

export class Gasto extends Movimiento {
  static ENCABEZADO_CSV = ['ID', 'Fecha', ...];
  metodoPago;
  categoria;
  constructor(datos) {
    super(datos);
    this.metodoPago = datos.metodoPago;
    this.categoria = datos.categoria;
  }
  get tipo() {
    return 'GASTO';
  }
  toJSON() {
    return { ...this.camposBase(),
             metodoPago: this.metodoPago,
             categoria: this.categoria };
  }
}
//# sourceMappingURL=Gasto.js.map
```

Transpilado por tsc — dist/src/dominio/Gasto.js

Figura 3. Snapshot del proceso de transpilación: los tipos, interfaces e importaciones de solo tipos desaparecen; la clase y la herencia se conservan como JavaScript estándar (ES2022).

III-F. Frontend en TypeScript

El cliente se implementó en la carpeta `client/` y se transpila a módulos ES que el navegador carga con `<script type="module" src="/js/main.js">` [11]. Se mantuvo la mejora progresiva: las páginas EJS funcionan sin JavaScript y los componentes agregan comportamiento. La base es la clase abstracta genérica `Componente<E extends HTMLElement>` (Fig. 12b), cuyo parámetro de tipo fija el elemento raíz (un `HTMLFormElement` para formularios, un `HTMLCanvasElement` para el gráfico). Los componentes implementados son:

- **ValidadorFormulario**: reglas tipadas equivalentes a los esquemas Zod del servidor, con mensajes por campo y atributo `aria-invalid`.
- **SimuladorPresupuesto**: consulta `GET /datos/presupuestos` y anticipa, antes de guardar, cómo quedará el presupuesto de la categoría (RF-10 en el cliente).
- **GraficoCategorias**: gráfico de dona en `<canvas>` con resaltado al pasar el puntero, alimentado con el `ResumenMensualDTO` que el servidor incrusta como JSON.
- **TablaOrdenable**, **VistaPreviaMonto**, **AvisoFlash**, **ConfirmacionAccion**, **MenuMovil** y **EnvioUnico**.

La comunicación con el servidor se encapsuló en `ClienteHttp`, cuyo método genérico `obtener<T>()` devuelve una *unión discriminada* `Resultado<T>`: el compilador impide usar `datos` sin comprobar antes `ok` (Fig. 4). Los nuevos endpoints `/datos/*` usan la misma cookie *httpOnly* de la sesión, por lo que el JavaScript del navegador nunca manipula el JWT. Los tipos de respuesta provienen de `contratos.d.ts`, el mismo archivo que usa el backend.

La Fig. 5 muestra el resumen mensual con el gráfico y el simulador en funcionamiento, y la Fig. 6 la validación en vivo y la tabla ordenada por monto.

IV. Modelado UML

Los diagramas siguen la notación UML 2.5.1 [12] y reflejan el código TypeScript implementado.

IV-A. Diagrama de casos de uso

La Fig. 7 presenta los actores *Visitante*, *Miembro* y *Administrador* (este último especializa al Miembro) y los casos de uso RF-01 a RF-14. En naranja se destacan los dos casos nuevos que aporta el frontend TypeScript: la simulación del presupuesto, que extiende a *Registrar gasto*, y la visualización del gráfico, que extiende a *Consultar resumen mensual*.

IV-B. Diagrama de componentes

La Fig. 8 muestra los componentes en tiempo de ejecución. En el navegador se ejecuta el JavaScript generado desde `client/`; en el servidor, el generado en `dist/`. El artefacto `contratos.d.ts` solo existe en tiempo de compilación y es importado con `import type` por el cliente y por el servidor. La API REST se ofrece como interfaz a clientes externos (Swagger UI, Postman) con autenticación *Bearer*.

IV-C. Diagrama de secuencia

La Fig. 9 describe el escenario principal del sistema migrado: registrar un gasto. En la primera fase el componente `SimuladorPresupuesto` consulta al servidor, autenticado con la cookie, y proyecta el porcentaje de ejecución en el navegador; en la segunda, el formulario se envía, el servidor valida con el esquema tipado, registra el gasto y calcula la alerta RF-10 con la clase `EjecucionPresupuesto`.

```
document.addEventListener('DOMContentLoaded', ()
    => {

    document.querySelectorAll('form[data-confirmar]')
    .forEach((form) => {
      form.addEventListener('submit', (e) => {
        if
    (!window.confirm(form.dataset.confirmar))
          e.preventDefault();
      });
    });
  const boton =
    document.querySelector('[data-menu]');
  const menu = document.getElementById('menu');
  boton?.addEventListener('click', () => {
    const abierto =
    menu.classList.toggle('abierto');
    boton.setAttribute('aria-expanded',
                       String(abierto));
  });
  // ... evita doble envío de formularios
});
```

Antes (JS) — public/js/app.js (completo)

```
export type Resultado<T> =
  | { ok: true; datos: T }
  | { ok: false; status: number; error: ErrorApiDTO['error'] };

export class ClienteHttp {
  readonly #cache = new Map<string,
    Promise<Resultado<unknown>>>();
  obtener<T>(ruta: string, parametros: Record<string, string> = {}
          ): Promise<Resultado<T>> { ... }
}
// En SimuladorPresupuesto (extends Componente<HTMLFormElement>):
const r = await clienteHttp
  .obtener<EjecucionPresupuestoDTO[]>('/datos/presupuestos', {
    mes });
if (!r.ok) return this.ocultar();      // obligatorio antes de
    usar datos
const p = r.datos.find((x) => x.categoria.id === categoriaId);
const proy = proyectar(p.gastado, monto, p.montoLimite);
this.mostrar(`... quedaría en ${proy.porcentaje}% ...`,
    proy.estado);
```

Después (TS) — client/nucleo/ClienteHttp.ts y SimuladorPresupuesto.ts

Figura 4. Snapshot comparativo del frontend: de un script sin tipos a componentes TypeScript que consumen la API con contratos compartidos y resultados tipados.

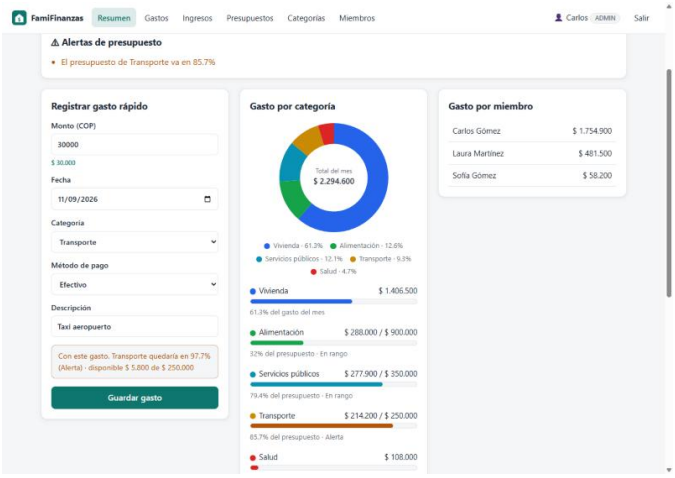


Figura 5. Resumen del mes: gráfico `GraficoCategorias` y simulación del presupuesto de Transporte antes de guardar un gasto (97,7 %, Alerta).

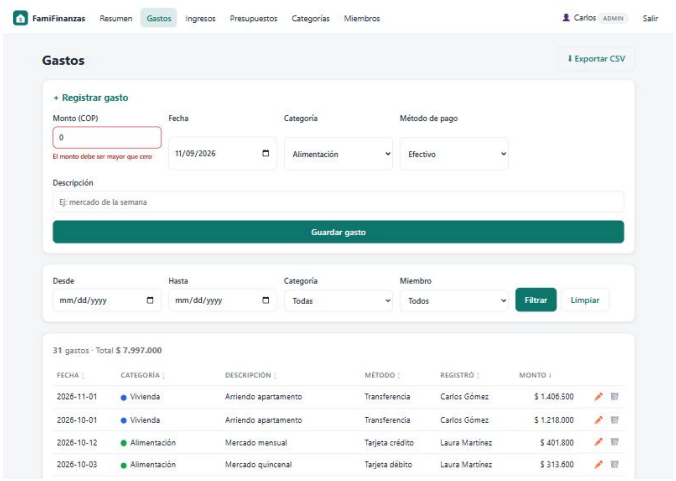


Figura 6. Gastos: `ValidadorFormulario` rechaza un monto de 0 y `TablaOrdenable` ordena por monto descendente.

IV-D. Diagramas de clases

La Fig. 10 presenta el modelo de dominio: herencia desde `Entidad<TDTO>`, la abstracción `Movimiento`, la realización de interfaces, la composición `Familia`–`Usuario`/`Movimiento` y las enumeraciones. La Fig. 11 muestra las capas de acceso a datos y de servicios, donde los genéricos permiten reutilizar la lógica común; y la Fig. 12 la jerarquía de errores y los componentes del cliente.

V. Resultados y Verificación

V-A. Pruebas automatizadas

Las pruebas se migraron a TypeScript y se ejecutan sobre el código transpilado. Las 26 pruebas de integración de la API y las 3 de las vistas web del Taller 2 pasan sin cambios en sus aserciones, lo que confirma que la migración preservó el comportamiento observable. Se agregaron 11 pruebas unitarias de la jerarquía de clases (herencia, polimorfismo, encapsulamiento, objetos de valor y errores) y una prueba de los endpoints del frontend que verifica el contrato JSON y el escape del JSON incrustado en la página (Tabla II).

Tabla II
Resultado de `npm test` (Node.js 22.17)

Archivo	Pruebas	Aprobadas
tests/api.test.ts (integración API)	26	26
tests/web.test.ts (vistas y /datos)	4	4
tests/dominio.test.ts (unitarias POO)	11	11
Total	**41**	**41**

V-B. Métricas de la migración

La Tabla III compara ambas versiones. El aumento de líneas se explica por las anotaciones de tipo, las interfaces y la

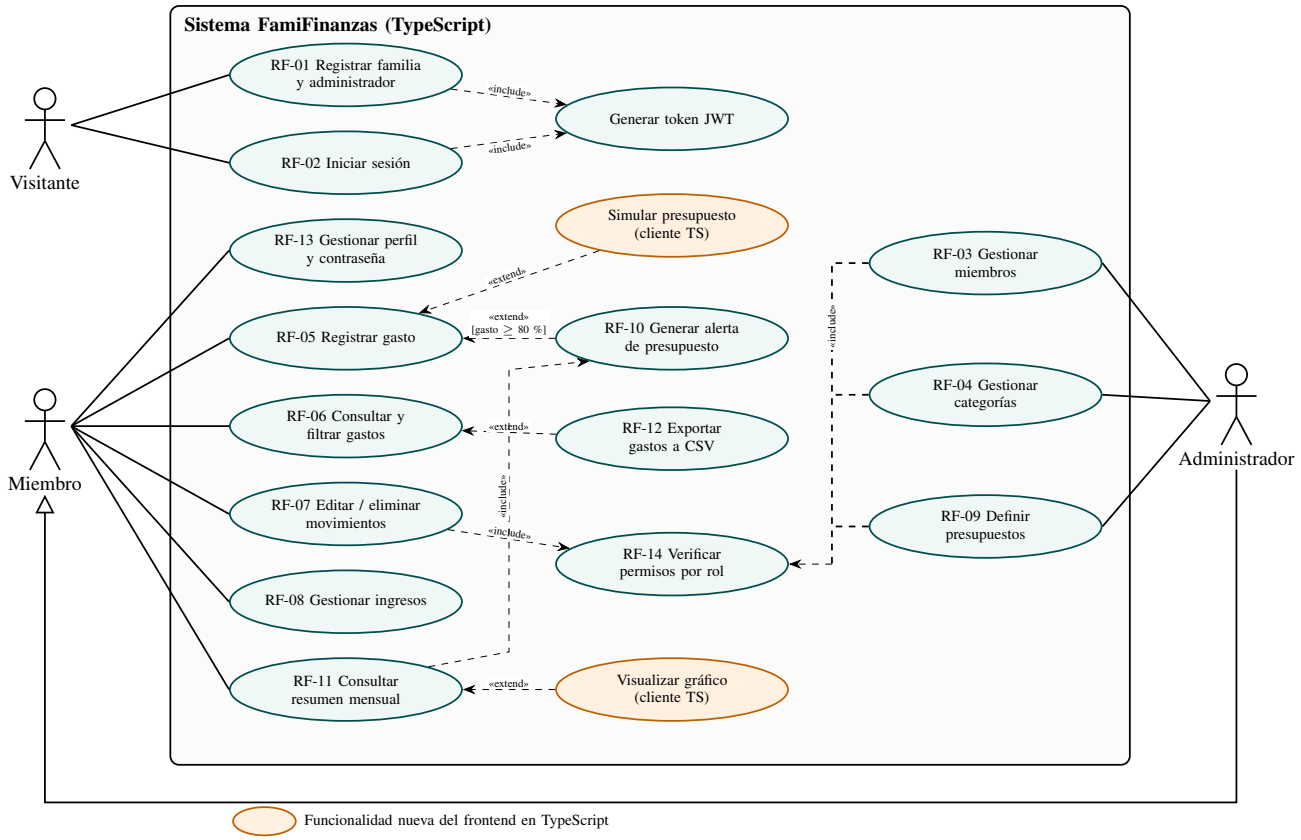


Figura 7. Diagrama de casos de uso de FamiFinanzas (versión TypeScript).

Tabla III
COMPARACIÓN ENTRE LA VERSIÓN JAVASCRIPT Y LA TYPESCRIPT

Métrica	**JS**	**TS**
Archivos del backend (`src/`)	25	46
Líneas del backend (con comentarios)	1 432	2 844
Archivos / líneas del frontend	1 / 25	12 / 612
Clases (backend + frontend)	1	47
Interfaces / alias de tipo / `enum`	0 / 0 / 0	62 / 29 / 3
Pruebas automatizadas	29	41
Errores de tipos al compilar (`tsc`)	—	0

documentación TSDoc, y por el nuevo frontend; a cambio, la lógica duplicada de repositorios y servicios se eliminó mediante herencia. El rendimiento (RNF-05) se midió con 10 000 gastos en la base de datos: el percentil 95 fue de 6,2 ms al listar gastos, 3,6 ms al registrarlos y 8,7 ms en el resumen mensual, muy por debajo del objetivo de 300 ms.

V-C. Ventajas observadas de TypeScript

- **Detección temprana**: errores de forma de datos, parámetros y nulos aparecen al compilar y en el editor, no en producción.
- **Contratos únicos**: `contratos.d.ts` y `z.infer` mantienen sincronizados API, servidor y cliente.
- **Diseño orientado a objetos verificable**: `abstract`, `implements`, `override` y los genéricos convierten el diagrama de clases en código que el compilador hace cumplir [9].
- **Mantenibilidad**: el autocompletado y la refactorización segura reducen el costo de cambio; los `Record<Enum, ...>` obligan a cubrir cada valor nuevo.
- **Sin costo en ejecución**: el JavaScript generado es equivalente al escrito a mano y el rendimiento no cambió.

VI. CONCLUSIONES

La migración de FamiFinanzas a TypeScript se completó en el backend y en el frontend sin alterar el comportamiento de la API, como lo demuestran las 29 pruebas originales que siguen pasando. El cambio no fue solo sintáctico: el modelo pasó de objetos literales a una jerarquía de clases con interfaces, clases abstractas, genéricos y encapsulamiento real, que coincide con el diagrama de clases y que el compilador hace cumplir. La configuración estricta y el uso de `import type` producen, tras la transpilación, JavaScript ES2022 limpio. En el cliente, TypeScript permitió construir componentes reutilizables que consumen la API con contratos compartidos y resultados tipados, lo que incorporó nuevas funcionalidades (simulación de presupuesto y gráfico) con menor riesgo. La principal lección es que una migración incremental respaldada por pruebas existentes permite adoptar TypeScript de forma segura, y que su mayor beneficio aparece cuando se aprovechan los tipos para expresar el diseño y no solo para anotar variables.

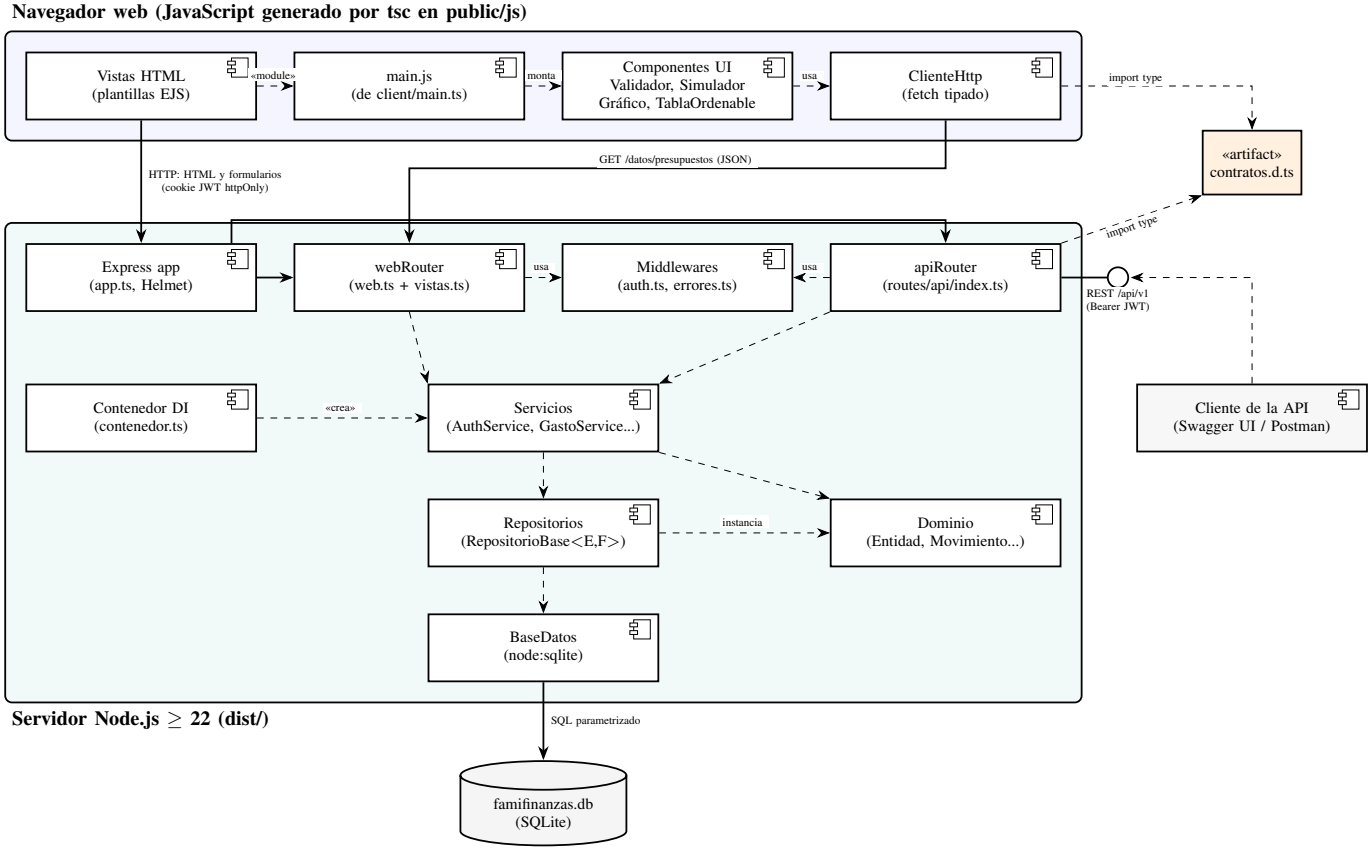


Figura 8. Diagrama de componentes: cliente TypeScript, servidor TypeScript por capas y contratos compartidos.

REFERENCIAS

[1] Microsoft, "The TypeScript Handbook," 2025. [En línea]. Disponible: https://www.typescriptlang.org/docs/handbook/intro.html

[2] G. Bierman, M. Abadi y M. Torgersen, "Understanding TypeScript," en *Proc. ECOOP 2014*, LNCS, vol. 8586, Springer, 2014, pp. 257–281.

[3] Z. Gao, C. Bird y E. T. Barr, "To type or not to type: Quantifying detectable bugs in JavaScript," en *Proc. 39th Int. Conf. Software Engineering (ICSE)*, 2017, pp. 758–769.

[4] Microsoft, "TSConfig Reference," 2025. [En línea]. Disponible: https://www.typescriptlang.org/tsconfig

[5] M. Jones, J. Bradley y N. Sakimura, "JSON Web Token (JWT)," IETF, RFC 7519, may. 2015.

[6] OpenJS Foundation, "SQLite — Node.js v22 Documentation." [En línea]. Disponible: https://nodejs.org/docs/latest-v22.x/api/sqlite.html

[7] C. McDonnell, "Zod: TypeScript-first schema validation with static type inference." [En línea]. Disponible: https://zod.dev

[8] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, EE. UU.: Addison-Wesley, 2002.

[9] E. Gamma, R. Helm, R. Johnson y J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, EE. UU.: Addison-Wesley, 1994.

[10] Ecma International, *ECMA-262: ECMAScript 2022 Language Specification*, 13.ª ed., jun. 2022.

[11] Mozilla, "JavaScript modules," MDN Web Docs. [En línea]. Disponible: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules

[12] Object Management Group, *OMG Unified Modeling Language (OMG UML), Version 2.5.1*, formal/17-12-05, dic. 2017.

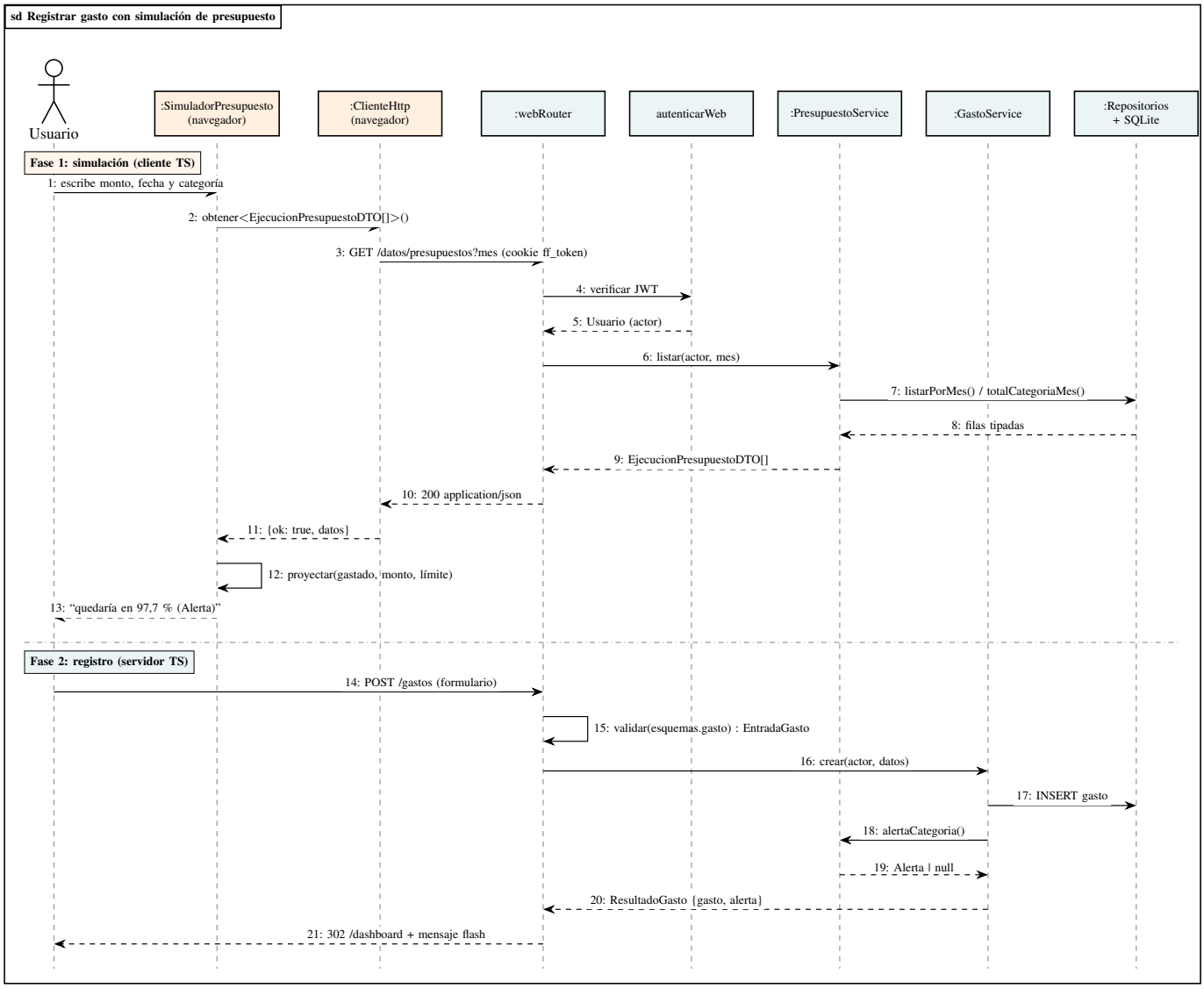


Figura 9. Diagrama de secuencia: registrar un gasto con simulación del presupuesto en el cliente TypeScript.

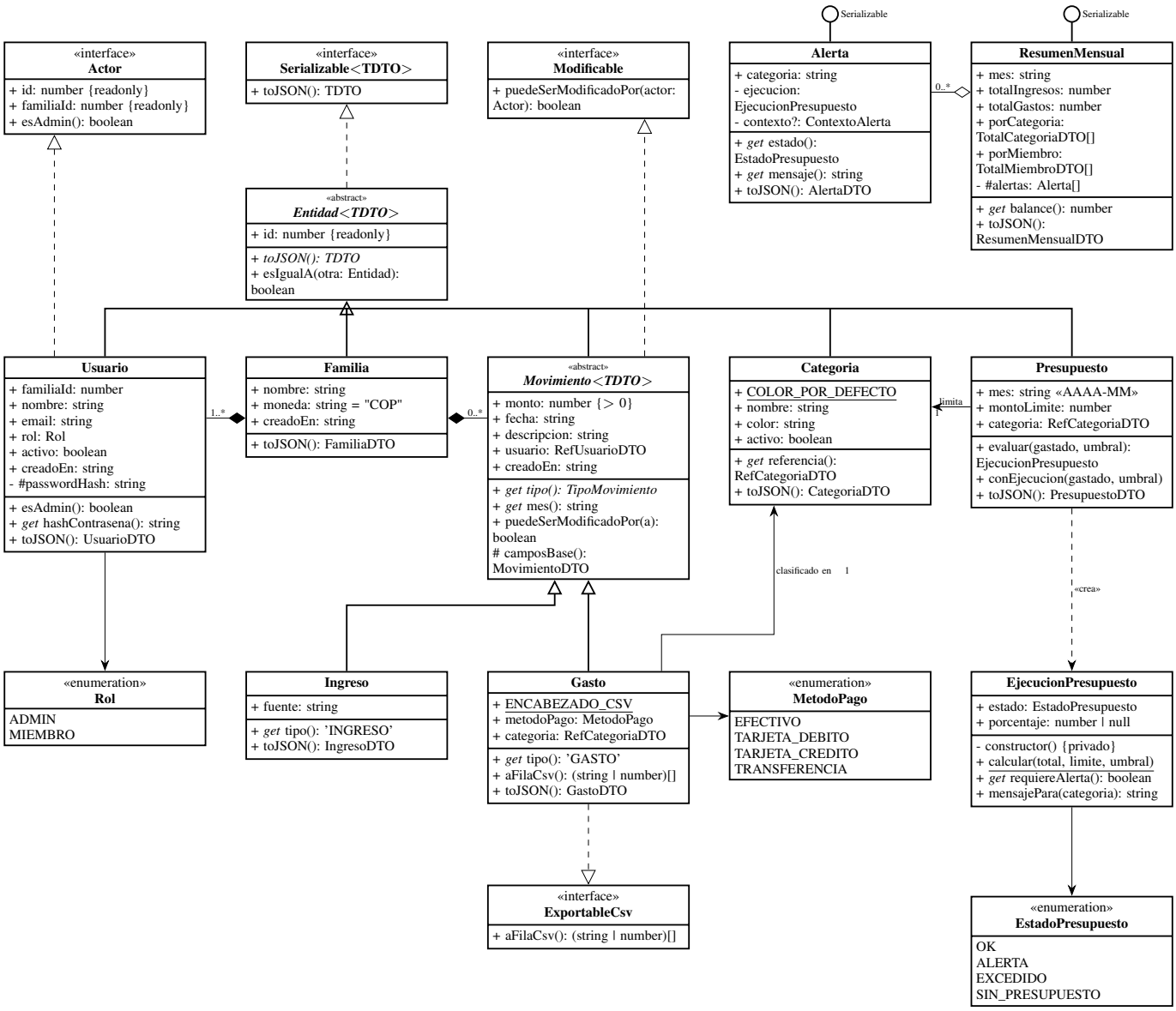


Figura 10. Diagrama de clases del dominio (`src/dominio`). Los nombres en cursiva son abstractos; # indica miembro protegido o campo privado de ECMAScript.

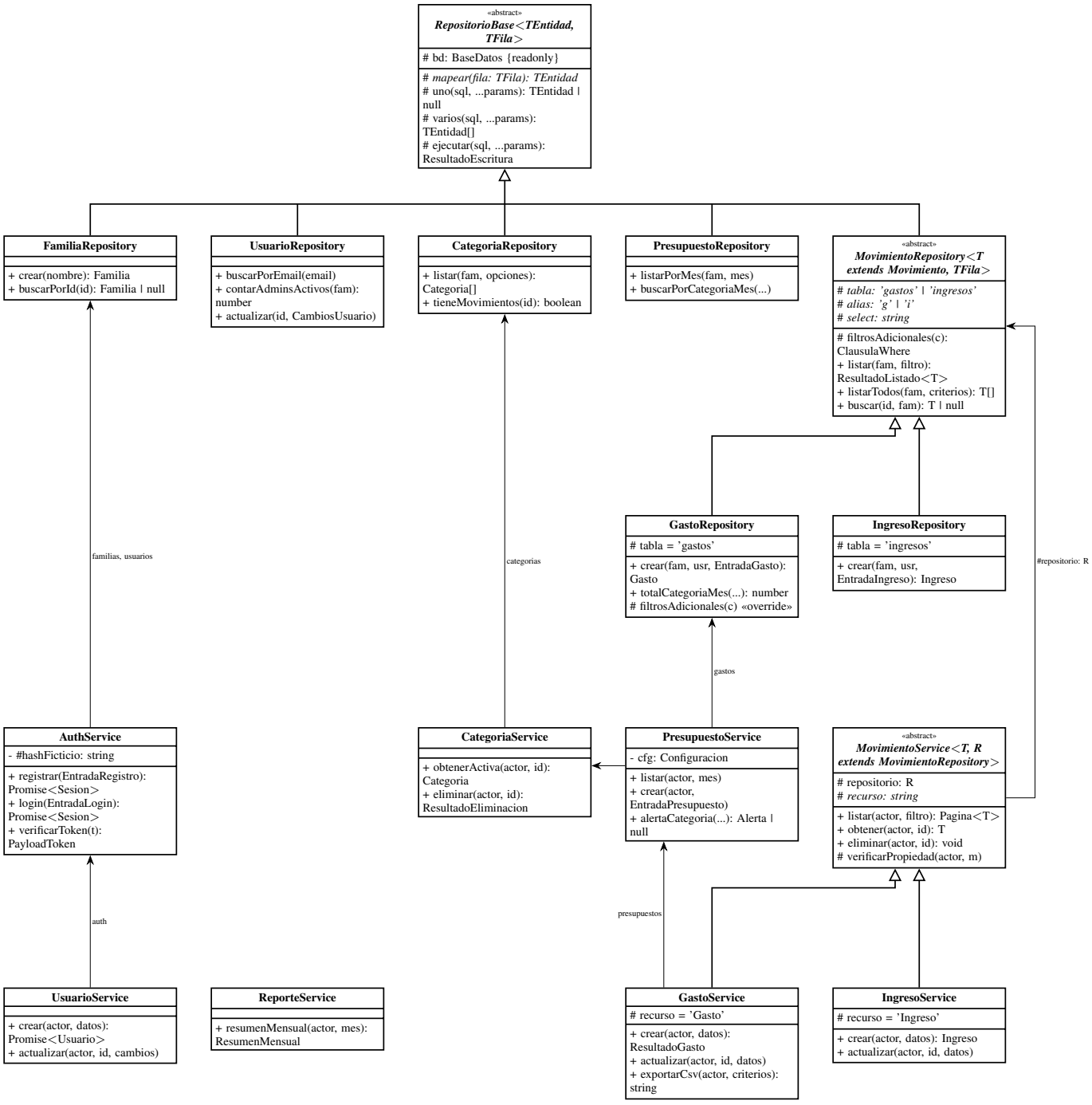
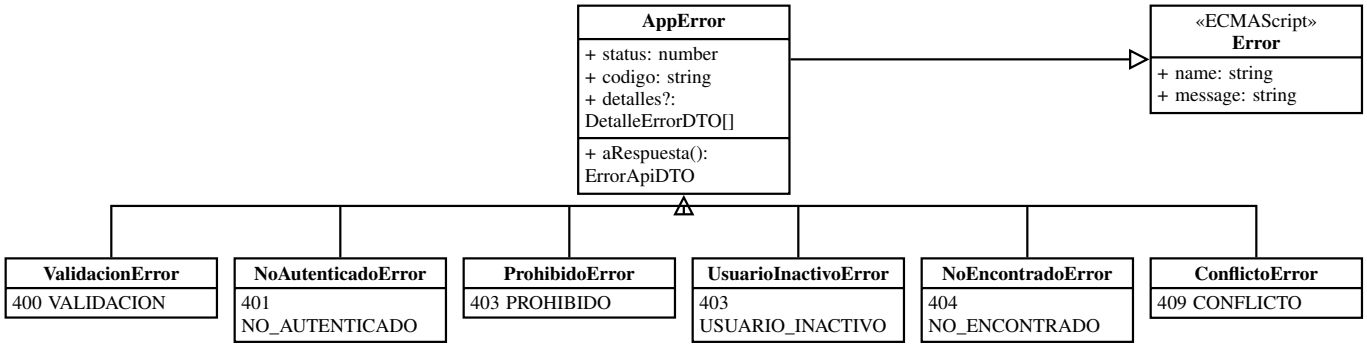


Figura 11. Diagrama de clases de las capas de repositorios y servicios: genéricos, clases abstractas y dependencias inyectadas por constructor.

(a) Jerarquía de errores del backend



(b) Componentes del frontend TypeScript

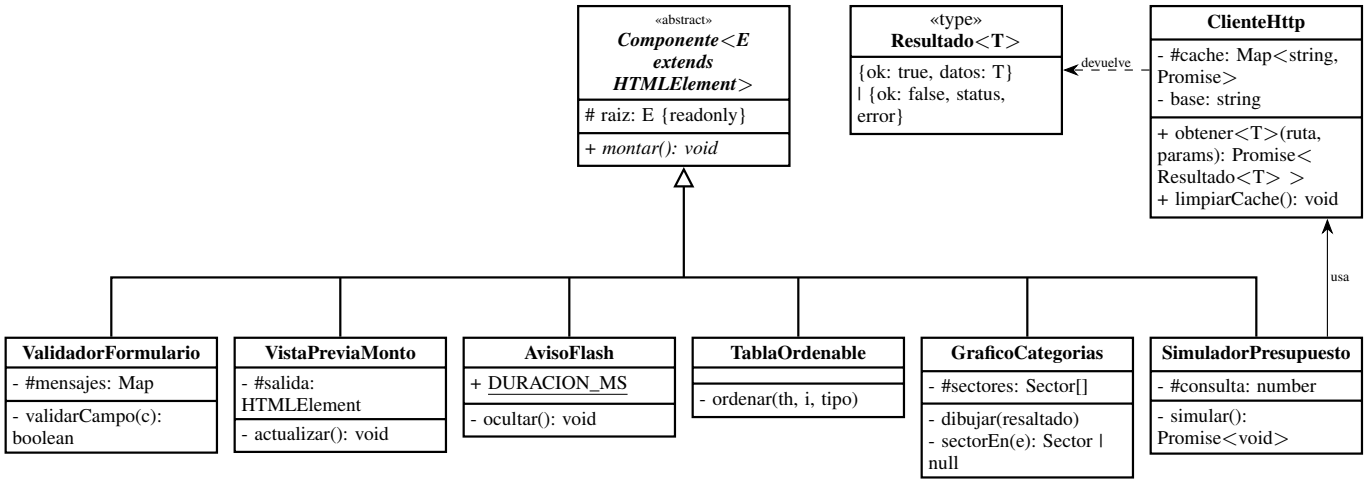


Figura 12. Diagrama de clases: (a) jerarquía de errores del backend; (b) componentes del frontend TypeScript (además: `ConfirmacionAccion`, `MenuMovil` y `EnvioUnico`).